

Reviewer Pack — Tropicalization of Kähler Manifolds and AC0 Circuit Threshold...

Entry 8563b8664090 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 09:45:47 UTC

Tropicalization of Kähler Manifolds and AC0 Circuit Threshold

Entry ID: 8563b8664090 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 09:45:47 UTC

1. Conjecture

Field A (mathematical branch): Complex Geometry (Kähler Manifolds) **Field B** (complexity object): Complexity Theory: AC0 Circuit Threshold

Statement:

[‘For a given input CNF formula with n variables, the tropicalization of its Kähler form, computed as the minimal rank of the associated sheaf on a toric variety, is logarithmic in the AC0 circuit threshold for the formula.’, ‘Equivalently, the AC0 circuit threshold k of the formula satisfies: $\log(n) \leq \min_rank(\text{Kähler_form_tropicalization}(\text{CNF})) \leq k * \log(n)$.’, ‘where $\text{Kähler_form_tropicalization}$ transforms a CNF into a tropical Kähler form by applying a constructive mapping from clauses to points on a suitable toric variety.’]

Rationale (proposer’s reasoning):

[‘Kähler manifolds provide a geometric framework that could potentially encode the complexity of boolean functions in a way that is amenable to analysis using tropical geometry.’, ‘The logarithmic relationship between the tropicalization of Kähler forms and AC0 circuit threshold might reveal a deep connection between complex geometry and computational complexity, suggesting new avenues for proving lower bounds.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e34b253ff788428

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 80% of the 30 random seeds, the ratio $\log(n)/\min_rank(\text{Kähler_form_tropicalization})$ is within $\pm 10\%$ of the measured AC0 circuit threshold k .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropicalization AND Kähler manifolds AND AC0 circuit threshold - minimal rank sheaf toric variety CNF formula - Kähler form tropicalization AC0 circuit threshold $\log(n) \leq \min_rank \leq k * \log(n)$

Top relevant hits considered: - [http://arxiv.org/abs/0806.0606v5] Toric Kähler metrics seen from infinity, quantization and compact tropical amoebas - [http://arxiv.org/abs/2303.09809v3] On tropical Hodge theory for tropical varieties - [http://arxiv.org/abs/2310.15367v1] Hodge theory for tropical fans - [http://arxiv.org/abs/0809.3148v2] Milnor fibers over singular toric varieties and nearby cycle sheaves - [http://arxiv.org/abs/math/0212044v3] Toric ideals, real toric varieties, and the algebraic moment map - [http://arxiv.org/abs/1303.4454v2] Characteristic classes of singular toric varieties - [s2:10.1145/2840728.2840734] The Complexity of DNF of Parities

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_cnf(n):
    cnf = []
    for _ in range(10 * n): # Generate enough clauses to ensure a non-trivial formula
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if all(clause[i] != -clause[j] for j in range(i)):
            cnf.append(tuple(clause))
    return cnf

def construct_toric_variety(cnf):
    # Simplified mapping from clauses to points on a toric variety
    return [sum(clause) for clause in cnf]

def compute_minimal_rank(points):
    n = len(points)
    if n == 0:
        return 0
    rank = 1
    for i in range(1, n):
        if all(points[i] != points[j] * (points[i][k] / points[j][k]) for j in range(i) for k in range(n)):
            rank += 1
    return rank

def ac0_circuit_threshold(cnf):
```

```

# Simplified AC0 circuit threshold calculation
return len(cnf)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    toric_variety = construct_toric_variety(cnf)
    min_rank = compute_minimal_rank(toric_variety)
    ac0_threshold = ac0_circuit_threshold(cnf)

    if min_rank == 0:
        return {
            "metric_name": "log(n) / min_rank(Kähler_form_tropicalization)",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "min_rank is zero"
        }

    ratio = math.log(n) / min_rank
    conjecture_holds = abs(ratio - ac0_threshold) <= 0.1 * ac0_threshold

    return {
        "metric_name": "log(n) / min_rank(Kähler_form_tropicalization)",
        "metric_value": ratio,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fe36dc8.py", line 79, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fe36dc8.py", line 48, in run_trial
    cnf = generate_cnf(n)
          ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0fe36dc8.py", line 22, in generate_cnf
    if all(clause[i] != -clause[j] for j in range(i)):
               ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was not possible to verify the conjecture's support or falsification. | next: Review and debug the test code to ensure it can run successfully and produce results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15090
2	preregistration	ollama_remote	glm4:latest	0	0	9849
3	novelty	ollama_remote	glm4:latest	0	0	8365
4	novelty	ollama_remote	glm4:latest	0	0	9781
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14785
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7322
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7442
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10179
9	judge	ollama_remote	glm4:latest	0	0	11613

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94427 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/8563b8664090.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8563b8664090.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8563b8664090.tar.gz` (if generated)